

De um workspace com

providers para uma

inteligência de software

de longo prazo



HIPÓTESE CENTRAL

O modelo é o motor. O Nocturne é o sistema que decide como pensar, agir, aprender e preservar o trabalho.

01 / POSICIONAMENTO

O produto não é um chat melhor

HOJE

Workspace + providers

Conversas, arquivos, Review, Build, Docs, Git e memória persistente, organizados em um mesmo lugar.

- O contexto depende da sessão
- O provider define o modo de pensar
- A memória é um apoio, não uma camada de decisão

umula contexto, coord

DESTINO

Nocturne Intelligence

Uma inteligência de software de longo prazo: orientada a projeto, consciente do estado e capaz de agir com limites claros.

- O projeto vira a unidade de continuidade
- Modelos são capacidades intercambiáveis
- Cada ação deixa evidência, estado e aprendizado

PRINCÍPIO DE ARQUITETURA

Codex deixa de ser a identidade do produto e passa a ser uma das capacidades cognitivas dele.

02 / VISÃO DE SISTEMA

Uma inteligência que compõe capacidades

A experiência é um ciclo: entender o objetivo, montar o contexto, escolher como pensar, executar com limites, verificar e atualizar o estado do projeto.



O diferencial não está em responder uma pergunta isolada.

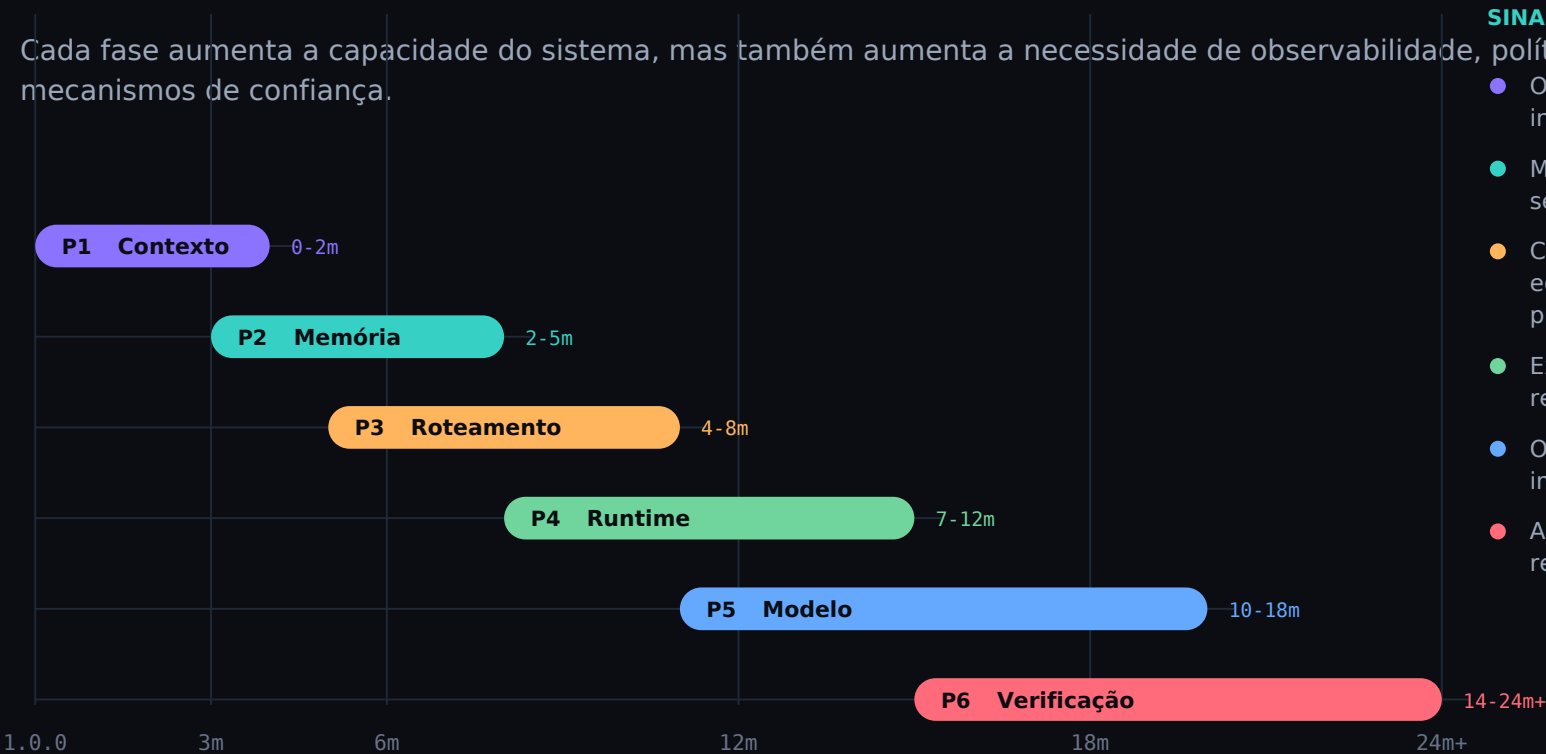
Está em fazer o projeto ficar mais compreensível, executável e confiável a cada ciclo.

03 / HORIZONTES

A evolução acontece em camadas

PÓS-1.0.0

Cada fase aumenta a capacidade do sistema, mas também aumenta a necessidade de observabilidade, políticas e mecanismos de confiança.



SINAIS DE PROGRESSO

- O contexto enviado pode ser inspecionado e corrigido.
- Memórias úteis sobrevivem às sessões e têm origem.
- Cada tarefa encontra o melhor equilíbrio entre qualidade, custo e privacidade.
- Execuções longas podem pausar, retomar e explicar o que fizeram.
- O sistema entende relações e impactos dentro do projeto.
- A autonomia cresce sem perder reversibilidade e confiança.

A ordem é deliberada: primeiro tornar o contexto observável; depois torná-lo durável; então automatizar decisões e ações.

04 / FASES 1-2

Contexto que entende e memória que permanece

P1 - Contexto + Awareness

Tornar o contexto uma unidade explícita, inspecionável e corrigível.

CAPACIDADES

- Montar ContextPack com intenção, arquivos, diffs, tarefas, restrições e decisões
- Awareness sobre objetivo ativo, recência, dependências, risco e bloqueios
- Mostrar o que foi incluído, omitido e por quê
- Orçamento de contexto, degradação controlada e trilha de decisão

SAÍDA DA FASE

80% das execuções têm contexto inspecionável; o usuário consegue corrigir o foco.

P2 - Memória viva

Transformar histórico em continuidade útil, com origem e validade.

CAPACIDADES

- Separar memória episódica, semântica, de projeto e operacional
- Registrar proveniência, confiança, escopo, uso, correção e expiração
- Fluxo de promoção: observação -> memória candidata -> memória aprovada
- Revisão, esquecimento e decaimento para evitar acúmulo de ruído

SAÍDA DA FASE

Memórias úteis sobrevivem entre sessões; cada fato importante pode ser rastreado até sua origem.

Primeiro o sistema aprende a ver o trabalho; depois aprende a não perdê-lo.

CONSTRUIR PARA ACUMULAR

05 / FASES 3-4

Roteamento de modelos e runtime próprio

P3 - Roteamento de modelos

Escolher o melhor motor para cada tarefa, sob políticas explícitas.

CAPACIDADES

- Classificar tarefa por tipo, sensibilidade, complexidade e urgência
- Matriz de capacidades por provider: qualidade, custo, latência, ferramentas e privacidade
- Perfis de roteamento por projeto, modo de trabalho e nível de risco
- Fallbacks, timeouts e comparação de resultados

SAÍDA DA FASE

Qualidade, custo e privacidade mensurados contra a linha de base de um único provider.

P4 - Runtime próprio de agentes

Executar planos como estado durável, não como uma sequência frágil de mensagens.

CAPACIDADES

- Run como máquina de estados: plano, ação, resultado, checkpoint e interrupção
- Ferramentas com permissões, escopo e contratos previsíveis
- Pausar, retomar e recuperar após falhas sem perder contexto
- Registro de eventos para explicar cada passo e cada decisão

SAÍDA DA FASE

Uma execução longa pode ser retomada e auditada sem depender da memória da conversa.

O Nocturne não precisa possuir todos os modelos; precisa possuir o ciclo de trabalho.

CONSTRUIR PARA ACUMULAR

06 / FASES 5-6

Modelo do projeto e autonomia progressiva

P5 - Modelo de conhecimento do projeto

Criar uma representação viva de como o projeto existe, muda e se relaciona.

CAPACIDADES

- Entidades: decisões, componentes, objetivos, riscos, convenções e dependências
- Relações ancoradas em evidências, recência e confiança
- Combinar grafo, busca vetorial e documentos estruturados
- Responder a impacto: o que muda se uma decisão, arquivo ou objetivo mudar?

SAÍDA DA FASE

O sistema responde perguntas de impacto com evidências e navegação até a origem.

P6 - Verificação + autonomia

Aumentar ação sem sacrificar controle, reversibilidade e confiança.

CAPACIDADES

- Políticas exigem testes, diff, revisão, confirmação e escopo
- Evidências coletadas antes de declarar uma tarefa concluída
- Incerteza explícita: confiança, hipóteses, bloqueios e necessidade de ajuda
- Escada L0-L4 de autonomia com rollback e auditoria

SAÍDA DA FASE

A autonomia aumenta porque o sistema prova mais, explica melhor e falha de modo seguro.

O objetivo final não é um agente que faz tudo; é um sistema que sabe quando agir e quando pedir ajuda.

CONSTRUIR PARA ACUMULAR

07 / ARQUITETURA

A arquitetura que compõe e aprende

O valor do Nocturne cresce quando seus objetos de trabalho têm forma, origem, estado e relações que podem ser reutilizadas por qualquer provider.

CAMADA DE EXPERIÊNCIA

Chat, Review, Build, Docs, memória, auditoria

CAMADA DE INTELIGÊNCIA

Awareness, ContextPack, Memory, Routing, Planning, Verification

CAMADA DE EXECUÇÃO

Execuções duráveis, ferramentas, permissões, checkpoints, eventos

CAMADA DE DADOS

Workspace, arquivos, Git, integrações, artefatos e credenciais

OBJETOS QUE TORNAM O SISTEMA COMPOSTO

Project

unidade de continuidade e escopo

Goal

intenção ativa e critério de conclusão

ContextPack

contexto explícito enviado a uma execução

Run

estado durável de uma ação ou plano

Memory

aprendizado com origem, confiança e validade

Evidence

prova, teste, diff, decisão ou fonte

Policy

limites de autonomia, dados e ferramentas

CICLO DE APRENDIZADO

OBSERVAR

→

MODELAR

→

PLANEJAR

→

AGIR

→

VERIFICAR

→

APRENDER

08 / FEEDBACK

Medir continuidade, não apenas respostas

O norte é manter trabalho confiável entre sessões e providers. As métricas devem revelar utilidade real, custo, risco e confiança.

NORTH STAR

Trabalho contínuo confiável por projeto

continuidade x utilidade x confiança

CONTINUIDADE

Sessões retomadas sem reexplicar o projeto.

- contexto reutilizado
- tempo até estado útil

CONTEXTO

Relevância e fidelidade do que foi enviado.

- contexto aceito / corrigido
- ruído e omissão

MEMÓRIA

Lembrar o que ajuda e esquecer o que atrapalha.

- recall útil
- correção e expiração

ROUTING

Qualidade adequada ao custo e à privacidade.

- qualidade por tarefa
- custo, latência e fallback

RUNTIME

Execuções longas e recuperáveis.

- runs retomáveis
- falhas recuperadas

CONFIANÇA

Ação com evidência e reversibilidade.

- cobertura de verificação
- rollback e incidentes

Instrumentar desde cedo: cada execução deve deixar estado, custo, evidência, resultado e motivo de falha.

09 / CONFIANÇA

Local-first é uma propriedade do sistema

A autonomia só é sustentável quando o usuário entende os limites, controla o que sai do workspace e consegue desfazer o que foi feito.

LOCAL-FIRST

Armazenar e processar localmente por padrão; externalizar apenas por decisão explícita.

FRONTEIRA EXPLÍCITA

Toda chamada externa declara dados, provider, finalidade e política aplicada.

MENOR PRIVILÉGIO

Ferramentas e credenciais têm escopo mínimo, tempo limitado e permissões revogáveis.

PROVENIÊNCIA

Memórias, decisões e respostas apontam para origem, data, confiança e estado.

REVERSIBILIDADE

Mudanças têm diff, checkpoint, rollback e confirmação proporcional ao risco.

PORTABILIDADE

Estado do projeto é exportável; nenhum provider deve ser a única fonte de continuidade.

ESCADA DE AUTONOMIA

L0

explica

observa e responde

L1

propõe

planeja e pede aprovação

L2

executa

dentro de escopo
aprovado

L3

age com limites

atua sob políticas
definidas

L4

delegado

auditoria e reversão

Regra: subir um nível de autonomia exige evidência de segurança no nível anterior.

10 / EXECUÇÃO

Começar pelo que torna todo o resto possível

A primeira entrega pós-1.0.0 não precisa parecer uma IA autônoma. Ela precisa tornar o estado do trabalho visível, durável e mensurável.

0-30 DIAS

Fundação observável

- Definir os schemas de Project, Goal, ContextPack, Run e Evidence
- Registrar rastros de contexto, provider, custo, resultado e falha
- Estabelecer uma linha de base de qualidade, latência e custo

ARTEFATO

contratos + rastros

31-60 DIAS

Contexto e memória

- Entregar o inspetor de Awareness e ContextPack
- Criar fluxo de revisar, promover, corrigir e expirar memórias
- Medir recuperação útil e ruído em tarefas reais

ARTEFATO

inspetor + revisão de memória

61-90 DIAS

Roteamento e runtime

- Implementar matriz de capacidades e políticas de roteamento
- Criar primeira execução durável com pausa, retomada e eventos
- Adicionar verificação mínima e rollback de artefatos

ARTEFATO

política de roteamento + primeira execução

EVITAR NESTA ETAPA

Treinar um modelo fundacional, prometer autonomia ampla, acumular integrações ou criar uma memória opaca.

A vantagem nasce da continuidade verificável do projeto, não da superfície mais chamativa.